NiftyScout v4 — Complete Project Handoff

What This App Does

NiftyScout is a locally-deployed Python/Streamlit app that analyses ~160 Indian stocks across Large/Mid/Small caps and builds a 15-stock growth portfolio designed to beat the Nifty 50 index over a long-term horizon. It runs on Windows, costs ₹0, and uses free data from Yahoo Finance + Screener.in.

Tech Stack

- **Language**: Python 3.13
- **UI**: Streamlit (run with `python -m streamlit run app.py`)
- **Data**: yfinance (price/volume/fundamentals), Screener.in (scraped: ROCE, promoter holding, PEG, etc.)
- **Analysis**: pandas-ta for technical indicators
- **Storage**: SQLite (data/niftyscout.db) for caching + performance tracking
- **Mobile Access**: ngrok tunnel
- **Email**: Gmail SMTP with App Passwords
- **Charts**: Plotly

Project Folder

```
C:\Users\LENOVO\Downloads\trading hints app\
├── app.py                 # Main Streamlit dashboard (entry point)
├── config.py              # ALL settings: tickers, weights, thresholds,
allocation
├── scoring.py             # The brain — 3-funnel scoring engine + relative
strength
├── data_fetcher.py        # yfinance data download + SQLite caching + Nifty/VIX
fetch
├── screener_scraper.py    # Scrapes Screener.in for ROCE, promoter, PEG, 5Y
growth, etc.
├── backtester.py          # Simulates the scoring model on historical data
├── position_sizer.py      # Position sizing calculator + market regime detector
├── performance_tracker.py # Tracks active positions, monitors target/SL hits
├── email_notifier.py      # Sends HTML email with daily picks via Gmail
├── tunnel.py              # ngrok tunnel manager for mobile access
├── start.bat              # One-click launcher for Windows
├── requirements.txt       # Python dependencies
```

```
├── data/
│   └── niftyscout.db      # SQLite cache (auto-created)
└── README.md
```

Architecture: Three-Funnel Scoring

The app scores stocks differently based on their market cap tier:

Stock Universe (~160 stocks)

- **Large Caps** (Nifty 50, ~50 stocks): Anchors, scored for quality & cash flow
- **Mid Caps** (Next 50 + quality midcaps, ~67 stocks): Growth engine
- **Small Caps** (~22 stocks): Alpha boosters, high growth potential

Portfolio Allocation: 30 / 50 / 20

- 4 Large Cap picks (30% of capital)
- 7 Mid Cap picks (50% of capital)
- 4 Small Cap picks (20% of capital)
- Total: 15 stocks
- Rebalance: Quarterly (63 trading days)
- Drift threshold: ±10% before forced rebalance

Scoring Weights Per Funnel

Dimension	Large Cap	Mid Cap	Small Cap
Technical	15%	15%	20%
Fundamental	40%	30%	25%
Institutional	20%	15%	10%
Risk	15%	10%	10%
Relative Strength	10%	30%	35%

Fundamental Sub-Weights Per Funnel

Large Cap emphasizes: ROCE (15%), D/E (15%), ROE (12%), ICR (10%)
Mid Cap emphasizes: 5Y Sales Growth (15%), 5Y Profit Growth (15%), PEG (15%), PE vs Industry

(12%)
Small Cap emphasizes: 5Y Sales Growth (18%), 5Y Profit Growth (18%), PEG (12%), D/E (12%)

All 33 Parameters Scored

Technical Score (5 parameters)

1. RSI (14-day) — momentum
2. MACD crossover — trend direction
3. SMA 20/50 crossover — golden/death cross
4. 200 DMA position — long-term trend + % distance
5. Volume spike ratio — conviction (current vs 20-day avg)

Fundamental Score (12 parameters)

6. P/E ratio (absolute)
7. P/E vs Industry PE (from Screener.in) or vs Nifty PE
8. PEG ratio (from Screener.in)
9. P/B ratio
10. ROCE (from Screener.in)
11. ROE
12. Debt/Equity ratio
13. Interest Coverage Ratio (from Screener.in)
14. 5-year Sales Growth (from Screener.in)
15. 5-year Profit Growth (from Screener.in)
16. Promoter Holding % (from Screener.in)
17. Pledged % (from Screener.in)

Institutional Score (4 parameters)

18. OBV (On-Balance Volume) trend — accumulation/distribution proxy
19. 5-day price return
20. 20-day price return
21. Dividend yield bonus

Risk Score (3 parameters)

22. Beta

23. 20-day annualized volatility
24. Drawdown from 52-week high

Relative Strength (NEW in v4)

25. 3-month return vs Nifty 50 (excess return) — This is the key alpha parameter. Mid/small caps are heavily weighted on this.

Exit & Hold (8 calculated parameters, not scored)

26. ATR (14-day) — for volatility-adjusted targets
27. Stop Loss — 20-day low + ATR buffer, capped at -8%
28. Target 1 — 2x risk-reward
29. Target 2 — 3x risk-reward, capped at 52-week high
30. Risk:Reward ratio
31. Hold period estimate (days)
32. Entry price

Market Context (2 displayed)

33. India VIX — fear gauge Nifty 50 level + daily change

Quality Gate (10 binary checks)

Each stock checked against: ROCE≥18%, ROE≥15%, D/E≤0.5, Sales5Y≥12%, Profit5Y≥15%, Promoter≥50%, ZeroPledge, ICR≥5, PEG<1.5, PE<Industry. Passes if ≥6/10.

Key Functions in scoring.py

- `compute_technical_score(df, nifty_df)` → includes RS calculation
- `compute_fundamental_score(fundamentals, funnel)` → funnel-aware weights
- `compute_institutional_score(df, fundamentals)`
- `compute_risk_score(df, fundamentals)`
- `compute_exit_and_hold(df, fundamentals, tech_details)`
- `score_stock(ticker, price_df, fundamentals, nifty_df)` → composite
- `rank_stocks(price_data, fundamentals, nifty_df)` → returns (all_ranked, picks_by_tier)

Key Functions in data_fetcher.py

- `fetch_price_data(tickers)` → dict of {ticker: DataFrame}, cached in SQLite
- `fetch_fundamentals(tickers)` → dict of {ticker: dict}, cached
- `fetch_nifty_index()` → DataFrame, patches with .info for latest price
- `fetch_india_vix()` → DataFrame
- `log_picks(picks)` → saves to picks_log table
- `get_picks_history(days)` → DataFrame

Key Functions in screener_scraper.py

- `scrape_screener_data(ticker)` → dict with ROCE, promoter, PEG, etc.
- `fetch_screener_data(tickers)` → cached batch scrape
- `merge_fundamentals(yf_data, screener_data)` → combines both sources

Key Functions in position_sizer.py

- `calculate_position_size(capital, entry, stop_loss, ...)` → shares, investment, risk
- `size_portfolio(picks, capital, ...)` → sized picks + cash remaining
- `detect_market_regime(nifty_df, vix_df)` → regime name, multiplier, signals

Key Functions in backtester.py

- `run_backtest(price_data, fundamentals, ...)` → equity curve, trades, stats, alpha vs Nifty

Key Functions in performance_tracker.py

- `add_position(...)` → track a new position in SQLite
- `check_positions_against_prices()` → alerts for target/SL hits
- `get_performance_summary()` → win rate, avg return, etc.

Known Issues / Recent Fixes

1. **yfinance data lag**: Nifty index price lags by a day. Fixed in `fetch_nifty_index()` by also reading `.info` for `regularMarketPrice` and patching it in.
2. **Timezone mismatch**: yfinance returns tz-aware timestamps, Python datetime is tz-naive. Fixed in backtester.py with tz_localize.

3. **Delisted tickers**: TATAMOTORS.NS and ZOMATO.NS fail. Replaced with TRENT.NS and ETERNAL.NS.
4. **NaN crash in exit calc**: Some stocks have NaN in pct_change. Fixed with try/except + pd.isna checks.
5. **Streamlit HTML rendering**: Multi-line f-strings with indentation render as code blocks. Fixed by building HTML as concatenated strings.
6. **pct_change FutureWarning**: Fixed by using `pct_change(fill_method=None)`.

What Still Needs Work

- The backtest with the old v3 (Nifty 50 only, 3 stocks, 5-day rebalance) showed -22% vs Nifty's -2.7%. The v4 rebuild (160 stocks, 15 picks, quarterly rebalance, RS filtering) hasn't been backtested yet.
- Screener.in scraping can 404 for some tickers (renamed companies). May need ticker mapping updates.
- Some small-cap tickers in config may not have data on yfinance — these silently fail and get skipped.
- The email notifier works but hasn't been configured by the user yet (needs Gmail App Password).
- Performance tracker (position monitoring) needs the user to manually click "Track today's picks" to start monitoring.

How to Run

```
cd "C:\Users\LENOVO\Downloads\trading hints app"
python -m streamlit run app.py
```

Or double-click `start.bat`.

User's Goal

Build a portfolio that beats Nifty 50 over 10 years using a quantitative, multi-factor scoring model with automatic position sizing, quarterly rebalancing, and risk management. The user wants to make "considerable capital" — they're ambitious but realistic about risk.